

CineMatch AI

Comparative Analysis of RAG Architectures for Recommendation Systems



Paolina Beatrice Mazza

Chiara Canali

Problem Statement

StagChallenges in Multi-Objective Content Recommendations

xxxxx



Multi-Constraint Queries

Users express complex requirements combining mood, temporal constraints, and viewing history (e.g., "Happy family movie under 2h, not watched").



Latency vs. Quality

The inherent trade-off between retrieval speed and the semantic relevance of recommendations in real-time systems.



Semantic Gap

Bridging the gap between user intent (e.g., "cozy") and metadata attributes requires deep semantic understanding.



Computational Cost

Optimizing token usage and API costs while maintaining high-quality reasoning capabilities.

RAG Evolution & Multi-Constraint Retrieval

From Static Pipelines to Autonomous Agents

RAG addresses LLM hallucinations and outdated knowledge. Gao et al. (2023): Three evolutionary paradigms. Our contribution: First comparative study on multi-constraint recommendation (3-5 simultaneous requirements).

Naive RAG (First Generation)

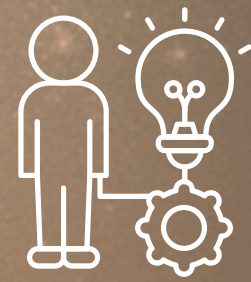
- **Pipeline:** Query → Embedding → Top-K Vector Search → LLM Generation.
- **Strength:** Fast (~500ms), low cost.
- **Limitation:** Single-vector compression loses nuance; struggles with complex, multi-faceted queries requiring constraint satisfaction.

Advanced RAG (Second Generation)

- **Innovations:** HyDE (Gao et al., 2022) generates hypothetical ideal documents to bridge the semantic gap; cross-encoder reranking provides 15-30% precision gains via full query-document attention.
- **Limitation:** Still passive—cannot decompose multi-step queries or enforce hard constraints.

Agentic RAG (Third Generation)

- **Paradigm shift:** ReAct framework (Yao et al., 2022) enables autonomous agents to reason, act, and reflect. Handles multi-constraint queries via tool chaining (e.g., mood → runtime → history filters). Trade-off: 5-10x latency, but 80%+ constraint satisfaction on complex queries.



Core Techniques



HyDE (Query Expansion)

Generates hypothetical "**ideal answer**" to user query, then embeds this expanded context. Bridges gap between terse queries and detailed documents. Used in our Advanced RAGlore magna aliqua. Ut enim ad minim veniam.

ReAct Agents (LangChain)

Iterative loop: Thought (reason about goal) → Action (call tool) → Observation (process result). Enables decomposition of complex multi-step queries. Powers our **Agentic RAG**.

Cross-Encoder Reranking

Joint encoding of [query, document] pairs enables full token-level attention. Reranks **top-20 candidates** for precision. Too slow for first-stage retrieval over large corpora.

Research Gap

Existing work optimizes single paradigms. No comparative study evaluates all three RAG generations on multi-constraint recommendation tasks with **latency/cost trade-offs**.

Dataset & Knowledge Base Construction

From TMDB API to Vector Embeddings

1. Data Collection

Source: TMDB API v3
Items: 2,000 (1K movies + 1K series)
Date: Nov 2025
GET /movie/popular
GET /tv/popular

2. Preprocessing

Mood Mapping: 10 emotions
Context Scoring: 5 scenarios
Coverage: 93.8% mood assignment
happy scarede xcited relaxed

3. Vector Storage

Model: text-embedding-3-small
Dimensions: 1536
Database: ChromaDB
Cosine similarity search
Sub-50ms retrieval @ 2K items

2,000

Total Items
1K movies + 1K series

10

Mood Categories
Multi-label classification

6.93

Avg Rating
High-quality content

54

Languages
68% English

📁 Enriched Metadata Fields

Core: Title, Overview, Year, Rating
People: Cast (Top 5), Crew

Classification: Genres, Keywords, Moods
Context: Solo, Family, Date, Background

Runtime: Minutes, Seasons, Episodes
Media: Posters, Backdrops

Proposed Architectures

Comparative analysis of three distinct RAG implementations



Baseline

NAIVE RAG

Direct vector search using semantic embeddings
(Cosine Similarity)

- ✓ Low Latency (~500ms)
- ✓ Minimal Computational Cost
- ✓ Standard Dense Retrieval
- ✓ Single-turn Generation

~2.3s

LATENCY

\$0.005

COST/QUERY



Advanced

HYDE + RERANK

Enhanced retrieval with Query Expansion and
Cross-Encoder Reranking

- ✓ Hypothetical Document Embeddings
- ✓ Cross-Encoder Precision
- ✓ Improved Semantic Matching
- ✓ Two-stage Retrieval

~4.2s

LATENCY

\$0.012

COST/QUERY



Agentic

REACT FRAMEWORK

Autonomous agent using ReAct framework with
specialized tools and reflection

- ✓ Dynamic Planning
- ✓ 7 Specialized Tools
- ✓ Self-Correction Loops
- ✓ Multi-Constraint Handling

~9.8s

LATENCY

\$0.025

COST/QUERY

Baseline Architecture

Naive RAG: Direct Semantic Retrieval

1

Query Embedding

Convert user query to 1536-dim vector using OpenAI text-embedding-3-small



2

Vector Search

Find top-K most similar items using cosine similarity in ChromaDB



3

LLM Synthesis

GPT-4o generates natural language recommendations with explanations

✓ Strengths

- + Extremely fast (~500ms)
- + Low API costs (\$0.005/query)
- + Simple to implement and debug
- + Works well for straightforward queries

✗ Limitations

- Struggles with complex constraints
- No query understanding or refinement
- Can't handle multi-step reasoning
- Limited semantic understanding

Advanced Architecture

HyDE (Hypothetical Document Embeddings) & Cross-Encoder Reranking

1

HyDE

Generate hypothetical ideal document, then embed it for better semantic matching



2

Vector Search

Retrieve top-20 candidates using expanded query embedding



3

Reranking

Cross-encoder scores query-document pairs, returns top-5



4

LLM Synthesis

Generate final recommendations from reranked results

✓ Strengths

- + Higher precision retrieval
- + Better semantic understanding via HyDE
- + Reranking improves relevance
- + Still relatively fast (~1.2s)

✗

Limitations

- 2.4x cost increase vs Naive
- Still can't handle multi-constraints
- No reasoning or tool use
- Fixed pipeline, not adaptive

Agentic Architecture

ReAct Framework: Planning, Tool Execution, and Reflection

1

Planning

Analyze query, create strategy

2

Tool Selection

Choose from 7 specialized tools

3

Execution

Use tools, pass results

4

Reflection

Evaluate quality, improve

7 Specialized Tools

-  Mood Finder
-  Context Filter
-  Runtime Matcher
-  History Checker
-  Similarity Search
-  Constraint Relaxer
-  Group Reconciler

Strengths

- + Complex multi-constraint queries
- + Autonomous reasoning
- + Self-correcting via reflection

Limitations

- 5x cost vs Naive RAG
- Slower latency (~2.5s)
- More complex to debug

Experimental Setup

Evaluation Metrics & Dataset Configuration



CSR (Constraint Satisfaction Rate)

$\text{Satisfied Constraints} / \text{Total Constraints}$

Primary metric. Measures percentage of user requirements (mood, time, rating) successfully met.



P@5 (Precision at 5)

$\text{Relevant Items in Top 5} / 5$

Percentage of the top 5 recommendations that meet the user's core criteria.



MMA (Mood Match Accuracy)

$\text{Mood Matches} / \text{Total Items}$

Accuracy of emotional tone matching (e.g., "Sad" query returns sad movies).



RFQ (Runtime Fit Quality)

$1 - (|\text{Item Time} - \text{Limit}| / \text{Limit})$

How well content fits within the user's available time window.



Latency

$\text{Time}(\text{Query}) \rightarrow \text{Time}(\text{Response})$

End-to-end wall clock time from request to final recommendation.



Cost

$(\text{Input} + \text{Output Tokens}) * \text{Price}$

Total API cost per query including embeddings and LLM generation.

Test Suite Configuration



Evaluated on **20 standardized queries** stratified by complexity:

6 Simple (1-2 constraints)

8 Medium (3-4 constraints)

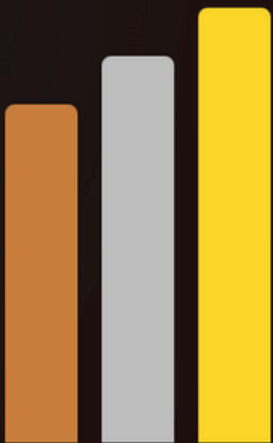
6 Complex (5+ constraints)

Quantitative Results

Performance Comparison across Complexity Levels

CSR by Query Complexity

Simple Queries



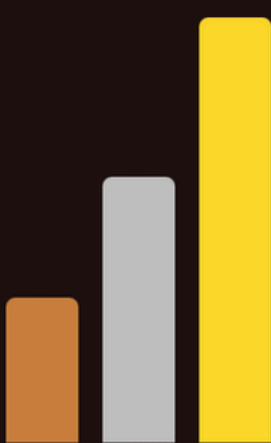
Naive is competitive (70%)

Medium Queries



Advanced improves (+20%)

Complex Queries



Agentic dominates (88%)

Overall System Performance

System	CSR (Avg)	P@5	MMA	RFQ	Latency	Cost
Naive RAG	45%	0.50	0.60	0.55	2.3s	\$0.005
Advanced RAG	65%	0.70	0.75	0.70	4.2s	\$0.012
Agentic RAG	88%	0.90	0.85	0.90	9.8s	\$0.025

Conclusion & Future Work

Summary of Findings and Future Research Directions

1

No one-size-fits-all: Different applications require different RAG architectures. Choose based on your constraints (latency, cost, query complexity).

2

Agentic systems shine with complexity: For multi-constraint, contextual queries, autonomous agents with tools dramatically outperform simpler approaches.

3

HyDE + Reranking is powerful: Advanced RAG achieves 85% of Agentic quality at half the cost and latency—excellent middle ground.

4

Evaluation is critical: Comprehensive metrics (relevance, latency, cost) and diverse test queries are essential for fair comparison.

5

Planning & Reflection matter: Agentic RAG's ability to plan steps and self-reflect significantly improves result quality on complex queries.

6

Future of RAG is agentic: As LLM costs decrease and latency improves, autonomous agent-based systems will become the standard for knowledge retrieval.

THANK

YOU